

Algoritmos y estructuras de datos

Arrays

Acceso directo

Con los arrays de acceso directo se puede establecer una relación entre dos datos que se ingresen, con una cantidad relativamente pequeña y conocida de variables. Por ejemplo, si tengo 100 empleados, quiero saber su sueldo y su legajo, con un solo vector basta, ya que en el mismo puedo cargar los sueldos y la posición del vector es la que hace referencia al legajo.

Paralelos

Carga de datos:

```
int cargarDatos(int vl[],int vs[],int cme)
{
    int leg,i=0;
    cout<<"Legajo (cero para finalizar) ";
    cin>>leg;
    while(leg!=0 && i<cme)
    {
        vl[i]=leg;
        cout<<"Sueldo I";
        cin>>vs[i];
        i++;
        cout<<"Legajo (cero para finalizar) ";
        cin>>leg;
    }
    return i;
}
```

Mejora para no permitir repetidos: hago una búsqueda secuencial que almaceno en una variable p. Luego pongo un if donde pregunto si p==-1. Si lo es, entonces sigo permitiendo el ingreso de datos, si no lo es, le pide que ingrese un nuevo dato.

```

int cargarDatos(int vl[],int vs[],int cme)
{
    int leg,i=0,p;
    cout<<"Legajo (cero para finalizar) ";
    cin>>leg;
    while(leg!=0 && i<cme)
    {
        p=secuencial(vl,i,leg); //AGREGADO 2
        if(p==-1) //AGREGADO 2
        {
            vl[i]=leg;
            cout<<"Sueldo ";
            cin>>vs[i];
            i++;
        }
        cout<<"Legajo (cero para finalizar) ";
        cin>>leg;
    }
    return i;
}

```

En acceso directo, para no permitir repetidos lo que preguntamos con if es si $v[\text{leg}-1] == 0$. Si es 0 entonces no había ingresado valor, si es distinto, le pedimos que ingrese nuevamente el dato.

```

void cargarDatos(int v[])
{
    int leg;
    cout<<"Legajo (cero para finalizar)"<<endl;
    leg=ingresarVal(0,100);
    while(leg!=0)
    {
        if(v[leg-1]==0)
        {
            cout<<"Sueldo"<<endl;
            cin>>v[leg-1];
        }
        else
        {
            cout<<"Ese legajo ya fue ingresado"<<endl;
            cout<<"Legajo (cero para finalizar)"<<endl;
            leg=ingresarVal(0,100);
        }
    }
}

```

Ordenamiento de vectores paralelos: cuando se establece un ordenamiento de vectores paralelos, lo que se modifica en un vector debe modificarse en el otro. Por ese mismo motivo se agrega en el código el intercambio respecto al otro vector.

```

void burbujeo(int vL[],int vs[],unsigned t)
{
    unsigned i=1,j;
    int aux;
    bool cambio;
    do
    {
        cambio=false;
        for(j=0;j<t-i;j++)
        {
            if(vL[j]>vL[j+1])
            {
                aux=vL[j];
                vL[j]=vL[j+1];
                vL[j+1]=aux;
                aux=vs[j];
                vs[j]=vs[j+1];
                vs[j+1]=aux;
                cambio=true;
            }
        }
        i++;
    } while(cambio);
}

```

Búsqueda del mayor elemento en un vector

Para buscar el mayor dentro de un vector existen dos tipos de funciones: aquellas que tienen en cuenta los ceros que estén dentro de un vector inicializado y las que no lo tienen en cuenta. Es importante recalcar que aquellas que no lo tienen en cuenta son mejores, en el sentido en que también pueden usarse para buscar el menor elemento.

Considerando los ceros:

```

int mayor(int v[],int t,int pos[],int &j) //fuera de ejercicio
{
    //devuelve el mayor y un vector con las posiciones
    int m;
    for(int i=0;i<t;i++)
    {
        if(i==0 || v[i]>m)
        {
            m=v[i];
            j=0;
            pos[j]=i;
            j++;
        }
        else
        {
            if(v[i]==m)
            {
                pos[j]=i;
                j++;
            }
        }
    }
}

```

Esta función lo que hace es recibir el vector donde se está buscando el mayor, su tamaño, un vector de posiciones donde se pondrán las posiciones donde se encuentre el mayor y por referencia, el tamaño de ese vector de posiciones.

Si i es 0 o se encuentra un elemento del vector que es mayor al que estaba definido como mayor hasta entonces, lo que se hace es definir al mayor como ese elemento, $j=0$ y se coloca en el vector de posiciones en j $pos[j]$ el valor de la posición donde se encontró el mayor, es decir, i y se aumenta $j++$. Si este no es el caso y el elemento del vector es igual, entonces lo que se hace es directamente colocar en $pos[j]$ el valor de la posición donde se volvió a encontrar ese mayor y se aumenta $j++$.

Sin considerar los ceros:

```
int mayor(int v[], int t, int pos[], int &j)
{
    //devuelve el mayor(sin considerar los ceros) y un vector
    int m=-1;
    bool pri=true;
    for(int i=0; i<t; i++)
    {
        if(v[i]!=0)
        {
            if(pri || v[i]>m)
            {
                pri=false;
                m=v[i];
                j=0;
                pos[j]=i;
                j++;
            }
            else
            {
                if(v[i]==m)
                {
                    pos[j]=i;
                    j++;
                }
            }
        }
    }
}
```

En este caso, la función es prácticamente igual que la anterior, con la diferencia de que se va a inicializar m en -1, se va a definir una bandera y se va a llevar a cabo la acción siempre y cuando el elemento del vector en cuestión sea distinto de 0. La booleana lo que permite es que la primera vez se inicialice m en el primer elemento distinto de 0 que encuentre. Si lo quisiera inicializar en $i==0$ no podría porque quizás la primera posición del vector es 0 y entonces nunca entraría al if.

Mayor único y posición:

```

1 int mayor(int v[],int t,int &pos) //fuera de ejercicio
2 { //devuelve el mayor único y su posición
3     int m=v[0];
4     pos=0;
5     for(int i=1;i<t;i++)
6         if(v[i]>m)
7         {
8             m=v[i];
9             pos=i;
10        }
11
12    return m;
13 }

```

En este caso se inicializa el mayor en la primera posición del vector y mismo la posición. Luego arranca a buscar desde la segunda posición y si encuentra alguno lo que hace es definir esa variable como el nuevo mayor y su posición.

Ordenamiento de vectores

Burbujeo

La idea de este método de ordenamiento es comparar cada elemento con el de al lado. Se ordena de menor a mayor o de mayor a menor (se pone un > o <). En la primer pasada se ubica el último elemento. En la segunda pasada, el ante último, y así sucesivamente. Es decir, en la primera pasada se ubica el elemento t-1, en la segunda t-2, en la tercera t-3, t-(n-1). Se necesitan t-1 pasadas para poder ordenar el vector.

Consta de dos ciclos for, uno con i, donde se cuentan las pasadas desde 1 a t sin incluir, y dentro de ese ciclo hay otro ciclo for con j, donde se compara el j con el j+1, este último va desde 0 hasta t-1 sin incluir (porque se llega hasta la anteúltima posición y se compara con la última)

Se utiliza una variable auxiliar para no perder el valor de la posición del vector que se va a desplazar.

```

void burbujeo(int v[], unsigned t)
{
    int i, j, aux;
    for(i = 1; i < t; i++)
    {
        for(j = 0; j < t-1; j++)
        {
            if(v[j] > v[j+1])
            {
                aux=v[j];
                v[j]=v[j+1];
                v[j+1]=aux;
            }
        }
    }
}

```



Se puede hacer dos mejoras a este método:

Mejora 1: La primera mejora consiste en que, como en cada pasada se ubica el t-n elemento, en las pasadas siguientes en vez de comparar todos los elementos otra vez, se puede comparar desde el primero hasta el t-n. Entonces el ciclo j va desde 0 hasta t-i.

```

void burbujeo(int v[], unsigned t)
{
    int i, j, aux;
    for(i = 1; i < t; i++)
    {
        for(j = 0; j < t-i; j++)
        {
            if(v[j] > v[j+1])
            {
                aux=v[j];
                v[j]=v[j+1];
                v[j+1]=aux;
            }
        }
    }
}

```

Mejora 2: Los vectores que se desean ordenar puede que estén casi totalmente ordenados, entonces no es necesario comparar muchos de sus elementos, es decir, se necesitan menos pasadas. Entonces, se intercambia el primer ciclo for por un while. Se debe hacer si o si una primera vuelta para saber si está ordenado, con un do while. Se utiliza una variable booleana inicializada en falso que indica si estaba ordenado o no. En caso de que no lo esté se efectúa un cambio. El do while se efectúa cuando i<t y la booleana es true.

```

void burbujeo(int v[], unsigned t)
{
    unsigned i=1, j;
    int aux;
    bool cambio;
    do
    {
        cambio=false;
        for(j=0; j<t-i; j++)
        {
            if(v[j]>v[j+1])
            {
                aux=v[j];
                v[j]=v[j+1];
                v[j+1]=aux;
                cambio=true;
            }
        }
        i++;
    }while(i<t && cambio);
}

```

Selección

Este método de ordenamiento consiste en, primero, buscar el menor en todo el vector y ponerlo en la primera posición, luego el segundo menor desde la segunda posición y ponerlo en dicha posición, y así sucesivamente.

Consta de dos ciclos for, el primero de ellos me va a decir cuantas veces se va a recorrer el vector para buscar el menor y desde donde se va a buscar el menor. Cuando i es 0 se busca el menor desde la posición 0 y cuando es 1 se busca desde la posición 1 y así sucesivamente.

Se inicializa el menor en la primera posición y se define la posición de ese menor. El ciclo j se empieza desde i+1 y se busca el menor desde ahí. Cuando sale del ciclo for j, el valor de la posición i se pone donde está el menor $v[pos] = v[i]$ para que no se pierda, y en la posición i se pone el menor $v[i] = menor$. Es decir, el mayor que quiero intercambiar lo pongo donde estaba el menor (más adelante en el vector) y el menor lo pongo en el lugar donde va a ir. Lo que se hace siempre es intercambiar la posición i con la posición del menor, en i se van poniendo los menores que se encuentran adelante.

```

void seleccion(int v[], unsigned t)
{
    unsigned i, j;
    int menor, pos;
    for(i=0; i<t-1; i++)
    {
        menor=v[i];
        pos=i;
        for(j=i+1; j<t; j++)
        {
            if(v[j]<menor)
            {
                menor=v[j];
                pos=j;
            }
        }
        v[pos]=v[i];
        v[i]=menor;
    }
}

```

Inserción

Este método de ordenamiento trabaja con una parte ordenada y otra desordenada. La metodología dice que desde la posición 0 hasta la posición $i-1$ el vector está ordenado y a partir de la posición i hasta la $t-1$, está desordenado. La idea es tomar el primero de la parte desordenada del vector e insertarlo ordenadamente en la parte ordenada del vector.

La primera posición del vector siempre va a estar ordenado, por eso el primer ciclo for arranca en $i=1$ hasta t . Tomo el i y lo inserto en la parte ordenada.

En una variable a se guarda el elemento que se debe insertar ($a=v[i]$) y ahora se empieza a ver dónde se debe insertar. Entonces se empieza a recorrer la parte ordenada al revés hasta que se encuentre el lugar donde poner a . Se inicializa una variable j en el último elemento de la parte ordenada, es decir $j=i-1$ y después se hace un ciclo while donde se pregunta si $j \geq 0$ y si $v[j] > a$ (los que sean mayores a a , cuando encuentro uno que no es mayor ya se ordenó). En la posición posterior al vector ordenado se pone el mayor, o sea $v[j+1]=v[j]$, se hace una "copia", se corre ese num y decremento j ($j--$). Cuando encuentra uno que no es mayor sale del ciclo while y guarda en $j+1$ el a , o sea, $v[j+1]=a$. Ese j va a indicar donde está el num que no es mayor.


```

void insercion(int v[], unsigned t)
{
    unsigned i;
    int a, j;
    for(i=1; i<t; i++)
    {
        a=v[i];
        j=i-1;
        while(j>=0 && v[j]>a)
        {
            v[j+1]=v[j];
            j--;
        }
        v[j+1]=a;
    }
}

```

Búsqueda de vectores

Por lo general, en una búsqueda no suele haber elementos repetidos. Hay dos tipos de búsqueda.

Secuencial

La idea de la búsqueda secuencial es que me devuelva la posición y en caso de que no lo encuentre, que me devuelva algo para saber que no está. Si lo encuentra, me devuelve la posición del elemento y si no, me devuelve -1.

La función recibe el vector, tamaño y lo que quiero buscar.

Por default no está, por eso defino pos=-1 y i=0. Utilizo un ciclo while que funciona si i<t y si pos===-1, es decir, el ciclo va a funcionar mientras no termine de recorrer el vector y mientras no lo encuentre. Dentro del ciclo while pregunto con un if si v[i]==bus. Si es correcto define a pos = i y devuelve pos. Si no entra al if lo que hace es incrementar i (i++) y sigue buscando. Si no encuentra nada en todo el vector directamente devuelve -1.

```

int secuencial(int v[], unsigned t, int bus)
{
    unsigned i=0;
    int pos=-1;
    while(i<t && pos===-1)
    {
        if(v[i]==bus)
            pos=i;
        i++;
    }
    return pos;
}

```

Otro método: En vez de preguntar por pos dentro del ciclo while, hace la comparación directamente en el ciclo while, que solo va a funcionar mientras $i < t$ y $v[i] \neq \text{bus}$ (elemento que se quiere buscar). O sea, mientras sea distinto el ciclo while va a aumentar i ($i++$). Cuando $v[i] == \text{bus}$ no va a entrar al while y, si $i == t$, o sea, se terminó el vector, devuelve -1 porque no lo encontró. En cambio, si no es así devuelve i , que es la posición.

```
int secuencial(int v[], unsigned t, int bus)
{
    unsigned i=0;
    while(i<t && v[i]!=bus) i++;
    if(i==t)
        return -1;
    else
        return i;
}
```

Binaria

La búsqueda binaria consiste en tomar el elemento del medio del vector. Si justo es ese elemento el que estaba buscando, ya está. Si no lo es, voy a comparar el elemento que estaba buscando con ese del medio. Si es menor, lo voy a buscar en la primera mitad del vector y sino en la segunda. Todo esto, **suponiendo que el vector está ordenado**. De esta forma se va descartando la mitad del vector sucesivamente, y cada vez trabajo con menos elementos.

Defino una variable pos inicializada en -1, una variable desde=0 que indica donde arranca la búsqueda y una variable hasta=t-1 que indica hasta donde se hace la búsqueda, y una variable medio.

Con un ciclo while se pregunta si $\text{desde} \leq \text{hasta}$ y si $\text{pos} == -1$. Dentro de ese ciclo while voy a definir el medio como $\text{desde} + \text{hasta} / 2$ y luego pregunto con if si el elemento del vector en la posición medio $v[\text{medio}] == \text{bus}$, o sea, si es igual al elemento que estoy buscando. De serlo se define $\text{pos} = \text{medio}$ y se devuelve pos. Si no lo es, se pregunta con un if si $\text{bus} < v[\text{medio}]$ y, en caso de serlo, se define un nuevo $\text{hasta} = \text{medio} - 1$, donde se descarta el medio anterior y se achica el rango donde se va a buscar dentro del vector. Si no lo es, se define un nuevo $\text{desde} = \text{medio} + 1$, donde también se achicará el rango de búsqueda, pero en este caso para la segunda mitad del vector.

```
int binaria(int v[], unsigned t, int bus)
{
    int pos=-1;
    unsigned desde=0, hasta=t-1, medio;
    while(desde<=hasta && pos==-1)
    {
        medio=(desde+hasta)/2;
        if(v[medio]==bus)
            pos=medio;
        else
        {
            if(bus<v[medio])
                hasta=medio-1;
            else
                desde=medio+1;
        }
    }
    return pos;
}
```

Vectores de Struct

Los vectores de Struct son como un “reemplazo” de los vectores paralelos. Un Struct es una estructura de datos donde no todos los datos son del mismo tipo, como sí pasa con los vectores. Dichos datos se los agrupa en una misma estructura porque pertenecen al mismo ente (por ejemplo: nombre, apellido, domicilio, sector donde trabaja, DNI de una persona determinada).

El struct se define tal que así: struct Nombre. Ese nombre no es una variable sino un tipo de dato. Entonces en el main, cuando se haga referencia al struct se define una variable de ese tipo.

```
struct Empleado
{
    unsigned nroLeg;
    string nomApe;
    float sueldo;
};
```

Esa variable va a contener todas las partes que tenga el struct. No se puede ingresar (ni mostrar ni comparar) una variable de dicho tipo, sino que se debe ingresar miembro por miembro poniendo: variable.parte.

```
int main()
{
    Empleado emple;
    cout<<"Ingresar nro de Legajo ";
    cin>>emple.nroLeg;
    cout<<"Ingresar nombre ";
    cin>>emple.nomApe;
    cout<<"Ingresar sueldo ";
    cin>>emple.sueldo;
    cout<<emple.nroLeg<<"-"<<emple.nomApe<<"-"<<emple.sueldo<<endl;

    return 0;
}
```

Se pueden usar en funciones

```
Empleado ingresarEmple()
{
    Empleado emple;
    cout<<"Ingresar nro de Legajo ";
    cin>>emple.nroLeg;
    cout<<"Ingresar nombre ";
    cin>>emple.nomApe;
    cout<<"Ingresar sueldo ";
    cin>>emple.sueldo;
    return emple;
}
```

```

void ingresarEmple(Empleado &emple)
{
    cout<<"Ingresar nro de Legajo ";
    cin>>emple.nroLeg;
    cout<<"Ingresar nombre ";
    cin>>emple.nomApe;
    cout<<"Ingresar sueldo ";
    cin>>emple.sueldo;

}

void mostrarEmple(Empleado e)
{
    cout<<e.nroLeg<<"-"<<e.nomApe<<"-"<<e.sueldo<<endl;
}

```

Si se quiere hacer referencia a un struct dentro de otro:

```

void ingresarEmple(Empleado &emple)
{
    cout<<"Ingresar nro de Legajo ";
    cin>>emple.nroLeg;
    cout<<"Ingresar nombre ";
    cin>>emple.nomApe;
    cout<<"Ingresar sueldo ";
    cin>>emple.sueldo;
    cout<<"Fecha de ingreso"<<endl;
    cout<<"dia ";
    cin>>emple.fechaIngreso.dia;
    cout<<"mes ";
    cin>>emple.fechaIngreso.mes;
    cout<<"anio ";
    cin>>emple.fechaIngreso.anio;
}

```

O bien se puede llamar a una función en el struct principal:

```

emple.fechaIngreso=ingresarFecha;

```

```

Fecha ingresarFecha()
{
    Fecha f;
    cout<<"dia ";
    cin>>f.dia;
    cout<<"mes ";
    cin>>f.mes;
    cout<<"anio ";
    cin>>f.anio;
    return f;
}

```

Se puede definir un vector de tipo struct:

```
int main()
{
    Empleado vecEmple[10];
    ingresarEmpleados(vecEmple, 10);
    return 0;
}
```

```
void ingresarEmpleados(Empleado v[], int ce)
{
    for(int i=0; i<ce; i++)
        v[i]=ingresarEmple();
}
```

Si se hacen cálculos se hace referencia a la parte del struct sobre el que se quiere hacer la operación:

```
float promedio(Empleado v[], int t)
{
    float sum=0;
    for(int i=0; i<t; i++)
        sum+=v[i].sueldo;
    return sum/t;
}
```

Para el ordenamiento se hacen los siguientes cambios en el burbujeo:

```
void ordenar(Empleado v[], unsigned t)
{
    unsigned i=1, j;
    Empleado aux;
    bool cambio;
    do
    {
        cambio=false;
        for(j=0; j<t-i; j++)
        {
            if(v[j].nroLeg>v[j+1].nroLeg)
            {
                aux=v[j];
                v[j]=v[j+1];
                v[j+1]=aux;
                cambio=true;
            }
        }
        i++;
    }while(i<t && cambio);
}
```

Para mostrar el vector de struct:

```

void mostrar(Empleado v[], int t)
{
    for(int i=0; i<t; i++)
        cout<<v[i].nroLeg<<"-"<<v[i].nomApe<<"-"<<v[i].sueldo<<endl;
}

```

Apareo

La idea de la metodología de apareo es trabajar con dos estructuras secuenciales a la par. Para ello necesito si o sí que los vectores estén ordenados por el mismo criterio. Deben tener por lo menos un miembro en común y tienen que estar ordenados por ese miembro. Se puede aparear un vector y un archivo, un vector y una lista, es decir, no necesariamente las estructuras deben ser del mismo tipo.

En el ejemplo que vamos a trabajar, cada estructura hace referencia a empleados que trabajaron en distintas sucursales (una sucursal por estructura). Entonces quiero un listado de los empleados que trabajaron en ambas sucursales.

En la función apareo se va a hacer un listado de todos los empleados que están en los dos vectores (sucursales), ordenado, y sin repetición. La función recibe los dos vectores con su respectivo tamaño y defino dos variables: i y j (ambas inicializadas en 0). Con i voy a recorrer el primer vector y con j, el segundo.

El apareo se hace mientras no termine de recorrer ninguno de los dos vectores. Esto se expresa con un while(i<tamañovector1 && j<tamañovector2).

Dentro del while se pregunta con un if si el vector 1 en el miembro que se quiere comparar es menor al vector 2 en el miembro que se quiere comparar. En caso de que lo sea se procesa con un cout, donde se muestra el menor vector. En este caso el menor vector sería el primero, por lo que se aumenta i++. Es decir, solo se avanza en el vector que se procesa.

Si el vector 1 en el miembro que se quiere comparar es igual al vector 2, se procesa uno de los dos (son iguales así que da igual cuál de ellos se procesa) y se aumentan tanto i++ como j++.

Si el vector 1 en el miembro que se quiere comparar es mayor al vector 2, se procesa el vector 2 y se aumenta j++.

```

void apareo(Empleado vs1[], unsigned t1, Empleado vs2[], unsigned t2)
{
    unsigned i=0, j=0;
    while(i<t1 && j<t2)
    {
        if(vs1[i].nroLeg<vs2[j].nroLeg)
        {
            cout<<vs1[i].nroLeg<<"-"<<vs1[i].nomApe<<endl;
            i++;
        }
        else
        {
            if(vs1[i].nroLeg==vs2[j].nroLeg)
            {
                cout<<vs1[i].nroLeg<<"-"<<vs1[i].nomApe<<endl;
                //cout<<vs2[j].nroLeg<<"-"<<vs2[j].nomApe<<endl;
                i++;
                j++;
            }
            else
            {
                cout<<vs2[j].nroLeg<<"-"<<vs2[j].nomApe<<endl;
                j++;
            }
        }
    }
}

```

Una vez que termina el ciclo while, si un vector es más chico que el otro, entonces van a haber elementos de uno de los vectores que no se va a procesar. Para procesarlo lo que se hace es lo siguiente:

```

    }
    for(int k=i; k<t1; k++)
        cout<<vs1[k].nroLeg<<"-"<<vs1[k].nomApe<<endl;
    for(int k=j; k<t2; k++)
        cout<<vs2[k].nroLeg<<"-"<<vs2[k].nomApe<<endl;
}

```

Se inicializa la variable k en el último elemento que se procesó y se muestra ese elemento hasta el tamaño de dicho vector.

Corte de control

Esta metodología es similar a la de Apareo, pero con la diferencia de que se van a mostrar los repetidos.

En el ejemplo lo que se va a repetir es el número de sucursal donde trabajó el empleado.

Se va a definir una función que va a recibir el vector y el tamaño.

Lo que va a ocurrir es que se va a recorrer el vector, y mientras la variable sea igual, se va a contar y cuando se termina con un empleado, se muestra y se empieza con el siguiente

empleado. Es decir, tengo dos ciclos: uno que va empleado por empleado y otro que va a ir dentro del mismo empleado.

Defino una variable *i* inicializada en 0, un contador que se va a resetear por cada empleado y una variable donde voy a guardar el legajo del empleado con el que estoy trabajando (*legA*). Luego se pone un *while* que se va a recorrer mientras *i*<*t* y dentro de ese ciclo se pone *cont*=0. Luego le asigno a *legA*=*v*[*i*].*nroLeg*, es decir, le asigno el legajo con el que voy a trabajar temporalmente. Ahora escribo un ciclo *do while* en el que se va a contar (*cont*++) y pasar al siguiente (*i*++) mientras el legajo con el que estoy trabajando sea igual al legajo siguiente y que *i*<*t*. Luego se muestra el empleado.

```
void corteControl(Empleado v[], unsigned t)
{
    int i=0, cont, LegA;
    while(i<t)
    {
        cont=0;
        legA=v[i].nroLeg;
        do
        {
            cont++;
            i++;
        }while(i<t && legA==v[i].nroLeg);
        cout<<legA<<" trabajo en "<<cont<<" sucursales"<<endl;
    }
}
```

Si tuviese que trabajar con un archivo o una lista, se utilizaría el mismo formato, pero cambiaría la parte de vectores. Otra versión del corte de control más específica para vectores trabaja con subíndices, *i* para el ciclo externo y *j* para el interno:

El contador lo pongo en 0 e inicializo *j* en *i*. Luego se pone un ciclo *do while* donde pongo *cont*++ y *j*++, es decir, se mueve *j*, mientras *j*<*t* y *v*[*i*].*nroLeg*=*v*[*j*].*nroLeg*.

Luego se muestra *v*[*j*].*nroLeg* con un *cout* y finalmente se pone *i*=*j* para que haga un salto y siga con el prox empleado.

```
void corteControl(Empleado v[], unsigned t)
{
    int i=0, j, cont;
    while(i<t)
    {
        cont=0;
        j=i;
        do
        {
            cont++;
            j++;
        }while(j<t && v[i].nroLeg==v[j].nroLeg);
        cout<<v[i].nroLeg<<" trabajo en "<<cont<<" sucursales"<<endl;
        i=j;
    }
}
```


Matrices

Las matrices son arreglos (colección de elementos homogéneos) que tiene dos posiciones, fila y columna. Pueden tener las dimensiones que sean.

Las matrices se declaran tal que así:

```
int main()
{
    int matriz[4][6];
    return 0;
}
```

Primero se escribe la cantidad de filas y después de columnas.

Carga y muestra de datos en una matriz

Las matrices se pueden llenar de forma secuencial o aleatoria. Si quisiera llenar la matriz de forma secuencial, la puedo llenar por fila o por columna.

Para llenar por columna utilizaremos la función cargarPorColumna. Dentro de las funciones, el primer corchete puede estar vacío, pero el segundo no, porque necesita saber la cantidad de columnas al menos. Voy a necesitar dos ciclos y dos subíndices. El ciclo externo maneja las columnas (j) y el interno, las filas.

```
void cargarPorColumna(int m[][6],int cf,int cc)
{
    for(int j=0;j<cc;j++)
    {
        for(int i=0;i<cf;i++)
        {
            cout<<"Ingresar elemento de la posicion "<<i<<" , "<<j<<endl;
            cin>>m[i][j];
        }
    }
}
```

Para mostrar, también se puede hacer por filas o columnas. En esta caso desarrollaremos la función mostrarPorFila, donde va a tener primero el ciclo que recorra las filas primero.

```
void mostrarPorFila(int m[][6],int cf, int cc)
{
    for(int i=0;i<cf;i++)
    {
        for(int j=0;j<cc;j++)
            cout<<m[i][j]<<" ";
        cout<<endl;
    }
}
```

Sumatoria de filas y/o columnas en una matriz

Para hacer sumatorias se pueden hacer también por filas o columnas. La función que haga ello me debe devolver un vector por referencia. En este caso desarrollaremos la función `sumatoriaFilas`. En esta función voy a definir una variable `s` que va a ser = 0 por cada recorrida de una fila, luego voy a recorrer la matriz por fila, sumando en cada fila las diferentes columnas. La posición donde voy a guardar el resultado `s` va a ser en un vector `v[i]`. Cada posición del vector es una fila.

```
void sumatoriaFilas(int m[][6],int cf,int cc,int v[])
{
    int s;
    for(int i=0;i<cf;i++)
    {
        s=0;
        for(int j=0;j<cc;j++)
            s+=m[i][j];
        v[i]=s;
    }
}
```

	0	1	2	3	4	5	
0	10	11	12	13	14	15	
1	20	21	22	23	24	25	
2	30	31	32	33	34	35	
3	40	41	42	43	44	45	

Después muestro el vector con dicha sumatoria

```
void mostrarVector(int v[],int t)
{
    for(int i=0;i<t;i++)
        cout<<v[i]<<endl;
}
```

Si quiero hacer la sumatoria por columnas es lo mismo pero el ciclo externo recorre las columnas:

```

void sumatoriaColumnas(int m[][6],int cf,int cc,int v[])
{
    int s;
    for(int i=0;i<cc;i++)
    {
        s=0;
        for(int j=0;j<cf;j++)
            s+=m[j][i];
        v[i]=s;
    }
}

```

Para cualquiera de las funciones se puede sumar directamente en el vector, sin utilizar la variable s:

```

void sumatoriaFilas(int m[][6],int cf,int cc,int v[])
{
    //int s;
    for(int i=0;i<cf;i++)
    {
        //s=0;
        v[i]=0;
        for(int j=0;j<cc;j++)
            v[i]+=m[i][j];
        //v[i]=s;
    }
}

```

Si lo inicializo previamente me ahorro de tener que poner v[i]=0.

Lo que se podría hacer también es recorrer el vector por fila pero sumar las columnas, tal que así. Lo importante es el subíndice del principio. En este caso el vector debe venir en 0 si o si, porque no puedo inicializarlo mientras lo recorro porque estoy sumando al revés de como recorro, es decir, recorro filas pero sumo columnas.

```

void sumatoriaColumnas(int m[][6],int cf,int cc,int v[])
{
    for(int i=0;i<cf;i++)
    {
        for(int j=0;j<cc;j++)
            v[j]+=m[i][j];
    }
}

```

Sumatoria general (mezclando filas y columnas)

Promedio de todos los elementos de la matriz:

```
float promedioMatriz(int m[][25], int cf, int cc)
{
    int suma=0;
    for(int i=0; i<cf; i++)
    {
        for(int j=0; j<cc; j++)
        {
            suma+=m[i][j];
        }
    }

    return (float)suma/(cf*cc);
}
```

Suma de la diagonal principal y la diagonal secundaria:

```
int sumarDiagP(int m[][25], int cf)
{
    int suma=0;
    for(int i=0; i<cf; i++)
    {
        suma+=m[i][i];
    }
    return suma;
}

int sumarDiagS(int m[][25], int cf)
{
    int suma=0;
    for(int i=0; i<cf; i++)
    {
        suma+=m[i][cf-1-i];
    }
    return suma;
}
```

Mostrar diagonal principal y secundaria:

```

void mostrarDiagP(int m[][25], int cf)
{
    for(int i=0; i<cf; i++)
    {
        cout<<m[i][i]<<endl;
    }
}

void mostrarDiagS(int m[][25], int cf)
{
    for(int i=0; i<cf; i++)
    {
        cout<<m[i][cf-1-i]<<endl;
    }
}

```

Suma triángulo superior e inferior:

```

int trianguloSup(int m[][25], int cf)
{
    int suma=0;
    for(int i=0; i<cf; i++)
    {
        for(int j=0; j<cf; j++)
        {
            if(i<j)
                suma+=m[i][j];
        }
    }
    return suma;
}

int trianguloInf(int m[][25], int cf)
{
    int suma=0;
    for(int i=0; i<cf; i++)
    {
        for(int j=0; j<cf; j++)
        {
            if(i>j)
                suma+=m[i][j];
        }
    }
    return suma;
}

```

Mayor elemento de una fila (mezclando filas y columnas)

```

void maxFilas(int m[][25], int cf, int cc, int v[])
{
    int mayorF;
    for(int i=0; i<cf; i++)
    {
        mayorF=0;
        for(int j=0; j<cc; j++)
        {
            if(m[i][j]>mayorF)
            {
                mayorF=m[i][j];
            }
        }
        v[i]=mayorF;
    }
}

```

Ahora vamos a resolver un ejercicio aplicando lo aprendido pero llenando la matriz de formas aleatoria.

```

/*
Un consorcio desea controlar la cobranza de las expensas en un edificio de 5 pisos
donde existen 15 departamentos en cada piso.
El valor de las expensas es de $ 3000.
Para registrar la cobranza se ingresa el número de piso (1 a 5)
y el número de departamento (correlativo de 1 a 15 en cada piso).
El ingreso finaliza con un piso igual a 0.
Al finalizar mostrar el dinero total recaudado y
una tabla con una X indicando aquellos departamentos
deudores de la siguiente manera:
TOTAL RECAUDADO
DEPARTAMENTOS DEUDORES
PISO    DPTO 1 DPTO2 ... DPTO 15
PISO 1   X
PISO 2   X     X
PISO 3   X
PISO 4   X     X
PISO 5   X
*/

```

Para resolver el ejercicio vamos a definir una matriz de booleano, que va a ser inicializada toda en falso según la siguiente función.

```

void inic(bool m[][15],int cf, int cc)
{
    for(int j=0;j<cc;j++)
        for(int i=0;i<cf;i++)
            m[i][j]=false;
}

```

Luego lo que hacemos es una función registrarPagos donde se van a ingresar los datos validados entre dos valores (piso y departamento). Se realiza un ciclo while mientras el piso sea distinto a 0.

En la matriz en la fila piso-1 y columna dpto-1 voy a asignar un true

```
int registrarPagos (bool m[][15],int cf,int cc)
{
    unsigned piso,dpto;
    cout<<"Piso"<<endl;
    piso=ingresoEntero (0,cf) ;
    while (piso!=0)
    {
        cout<<"Dpto"<<endl;
        dpto=ingresoEntero (1,cc) ;
        m[piso-1][dpto-1]=true;
        cout<<"Piso"<<endl;
        piso=ingresoEntero (0,cf) ;
    }
}
```

Finalmente voy a desarrollar una matriz mostrarDeudores donde muestra el piso (i+1) y en cada piso recorre los departamentos donde, si m[i][j] es falso, muestra una X

```
}
void mostrarDeudores (bool m[][15],int cf,int cc)
{
    cout<<"      1   2   3   4   5   6   7   8   9   10  11  12  13  14  15 "<<endl;
    for(int i=0;i<cf;i++)
    {
        cout<<i+1<<" ";
        for(int j=0;j<cc;j++)
        {
            if(!m[i][j])
                cout<<" X ";
            else
                cout<<" ";
        }
        cout<<endl;
    }
}
```